

RAI-853: Advanced Robotics with ROS

Lec 03: Core Concepts

Instructor:

Dr. Khawaja Fahad Iqbal

Deputy HoD (Department of Robotics & AI, SMME, NUST)

Scientific Director (Intelligent Robotics Lab, NCAI)



- **Nodes**
- **Topics**
- **Services**
- **Actions**
- **Parameters**
- **Launch Files**

- **Nodes**
- Topics
- Services
- Actions
- Parameters
- Launch Files

- You will now start your first ROS 2 programs. As you will see, **a ROS 2 program is called a node.**
- In this lecture, we will focus on what's inside a node, what does it do, how do nodes communicate with each other, and how do you configure nodes and start several of them at the same time.
- We won't write any code yet but instead focus on discovering the concepts through hands-on experimentation, using existing demos that were installed along with ROS 2.

- To understand what a node is, we will simply run one and make some observations using some of the most useful ROS 2 tools.
- Let's discover your first ROS 2 tool, and probably the most important one: the `ros2` command-line tool. You will use this tool all the time in your ROS projects.
- To start a node, we have to follow this template:
`ros2 run <package> <executable>`.

- As we will see later, nodes are organized inside packages. That's why we first need to specify the package name where the node is and the executable name for that node.
- As we installed ROS Desktop, a lot of demo packages are already included, for example, `demo_nodes_cpp`.
- In Terminal 1, start the talker node from the `demo_nodes_cpp` package:

```
ros2 run demo_nodes_cpp talker
```

- After we run this command, the node starts. To stop it, simply press **Ctrl + C** in the terminal where the node is running.
- So, what happened here? From what we can observe, this node is simply a program that will print a log in the terminal every second.

[INFO] [1710223859.331462978] [talker]: Publishing: 'Hello World: 1'

[INFO] [1710223860.332262491] [talker]: Publishing: 'Hello World: 2'

[INFO] [1710223861.333233619] [talker]: Publishing: 'Hello World: 3'

- Now, keep the node alive, or start it again if you stopped it.
- In another terminal (Terminal 2), let's start a different node, which is the listener node from the same package:

ros2 run demo_nodes_cpp listener

- This node is also a simple program that will print some logs in the terminal.

- However, as we can see, when the two nodes are running (talker and listener), whatever is printed on the talker seems to also be received on the listener, which then prints it.

```
[INFO] [1710224252.496221751] [listener]: I heard: [Hello World: 9]
```

```
[INFO] [1710224253.497121609] [listener]: I heard: [Hello World: 10]
```

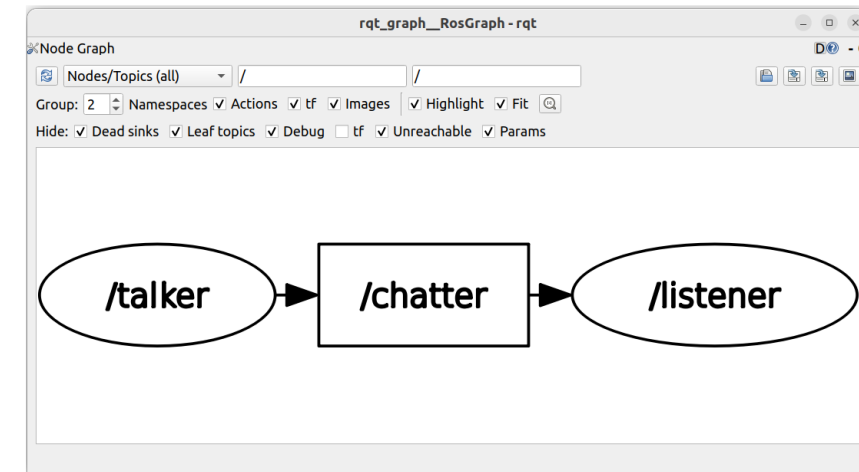
```
[INFO] [1710224254.495878769] [listener]: I heard: [Hello World: 11]
```

- In this example, we have two nodes running, and we can clearly see that they communicate with each other. If you stop the talker node, you will see that the listener node stops printing logs as well. When you restart the talker, the listener starts printing what the talker is “sending.”

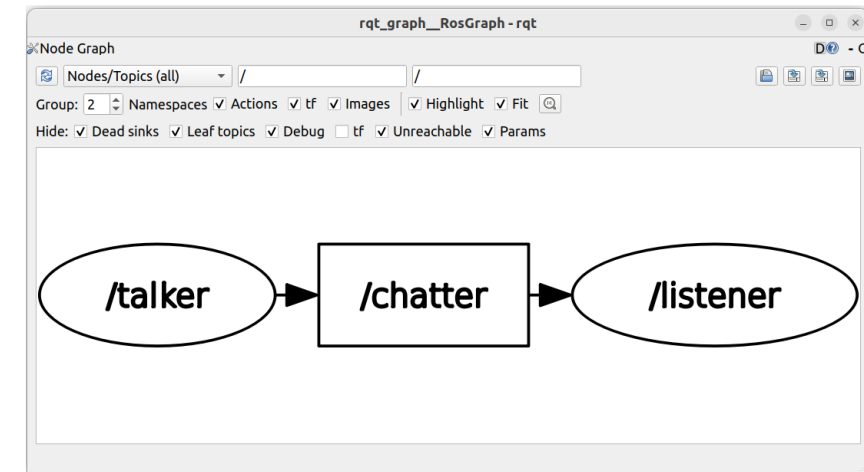
- rqt_graph will show you all running nodes with a nice visual.
- Keep the 2 nodes alive (Terminals 1 and 2) and start rqt_graph in Terminal 3. The command is identical to the tool name:

rqt_graph

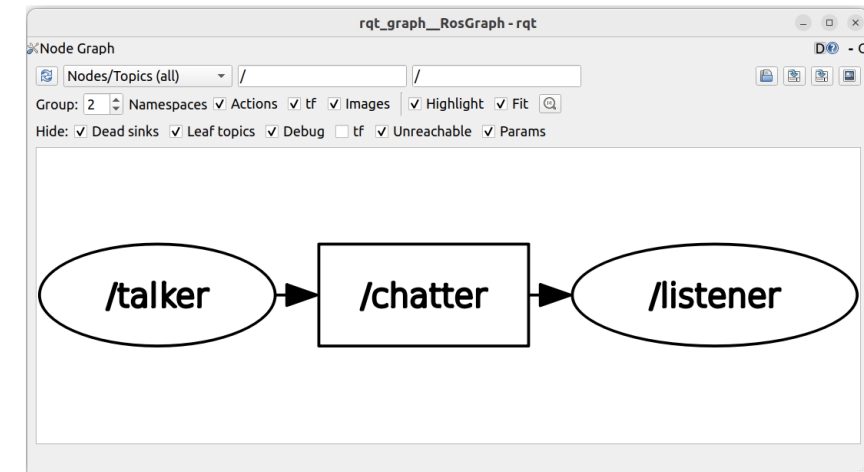
- This will open a new graphical window, where you should see the two nodes. If you don't see anything, make sure both nodes are running, and refresh the view by clicking on the button with a refresh icon, in the top-left corner.
- You can also select Nodes/Topics (all) from the top left drop-down menu.



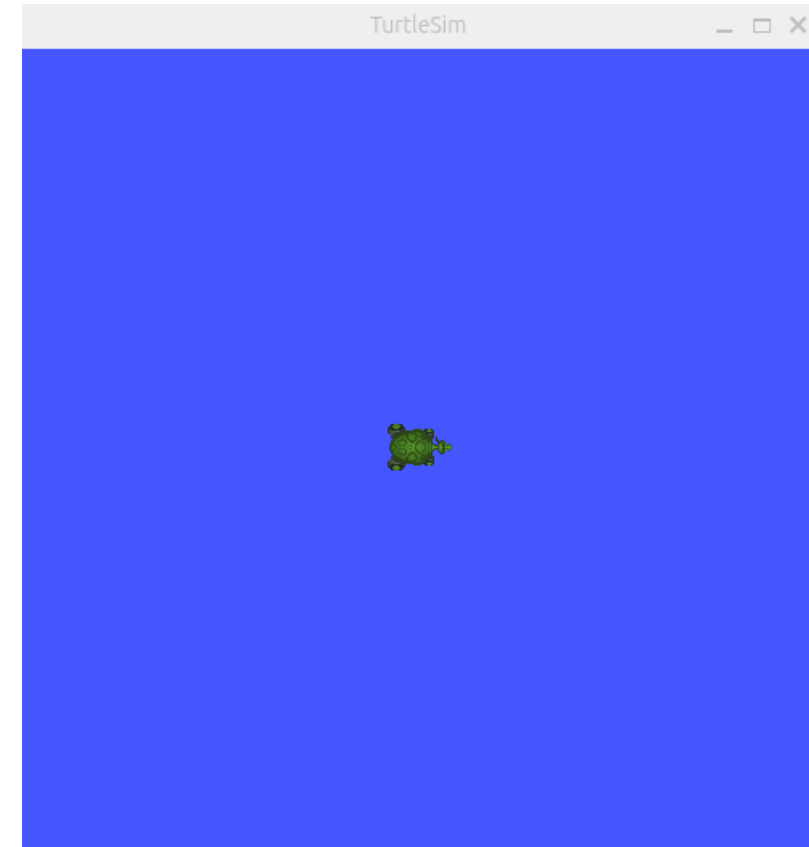
- Here, we can see that both nodes are up and running (nothing new for now), but we also see an arrow going from the talker node to a box, and another one from that box to the listener node.
- This is the **ROS 2 communication** that allows the talker node to send some data—here, some text—to the listener node.
- What we can conclude for now is that we started two different ROS programs (nodes) in two different terminals, using the `ros2 run` command.



- It seems that those **two programs are communicating with each other**, and we can confirm that with rqt_graph.
- Before we go further and look at what kind of ROS communication it is, let's run another set of nodes.



- Now, stop all existing nodes (press Ctrl + C in each terminal), and let's start again with some other nodes.
- In Terminal 1, run the following command:
ros2 run turtlesim turtlesim_node
- You'll see a few logs, but more importantly, you will get a new window with a blue background and a turtle in the middle.
- This turtle represents a (very simplified) simulated robot that moves in 2D space.



- In Terminal 2, start this second node:

```
ros2 run turtlesim turtle_teleop_key
```

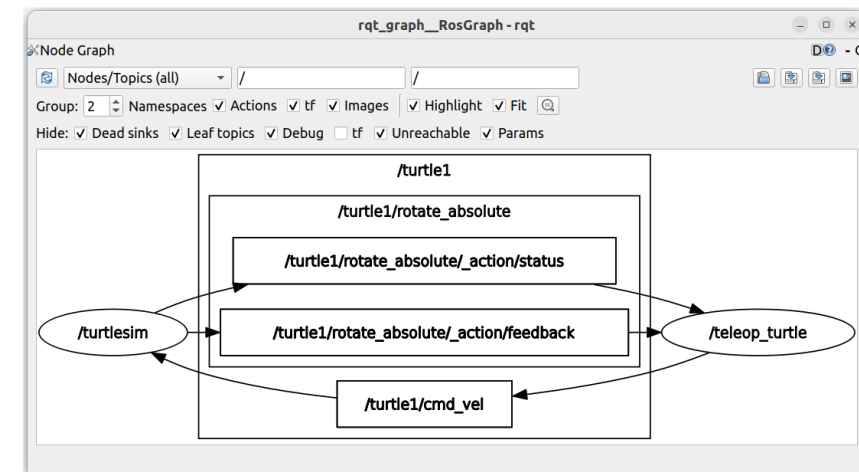
- After you see this, make sure Terminal 2 is selected, and **use the arrow keys** (up, down, left, right).
- When doing this, you should see the turtle robot moving.



- We can already guess that the second node we started (with turtle_teleop_key) is reading the keys you press on the keyboard and sending some kind of information/command to the turtlesim node, which then makes the turtle robot move.
- To confirm that, start rqt_graph again on Terminal 3:

rqt_graph

- If needed, refresh the view a few times. Select Nodes/Topics (all), and you'll see something like this.
- We can find a node named turtlesim and another one named teleop_turtle, and we can clearly see that the two nodes are communicating with each other.



- What can we get from those two examples? As you can see, a ROS 2 node can be any kind of computer program that contains:
 - Instructions to print logs in the terminal
 - Graphical windows (2D, can also be 3D)
 - Hardware drivers and more
- A node, on top of being a computer program, also benefits from the ROS 2 functionalities: logs, communication with other nodes, and other features we will discover throughout this book.

- Nodes
- **Topics**
- Services
- Actions
- Parameters
- Launch Files

- Nodes communicate with each other using ROS 2 communication features.
- There are three types of communication: **topics, services, and actions**. We will discover all three of them, starting with topics.
- Stop all running nodes (Ctrl + C), and let's come back to our first example.

- In Terminal 1, input the following:

```
ros2 run demo_nodes_cpp talker
```

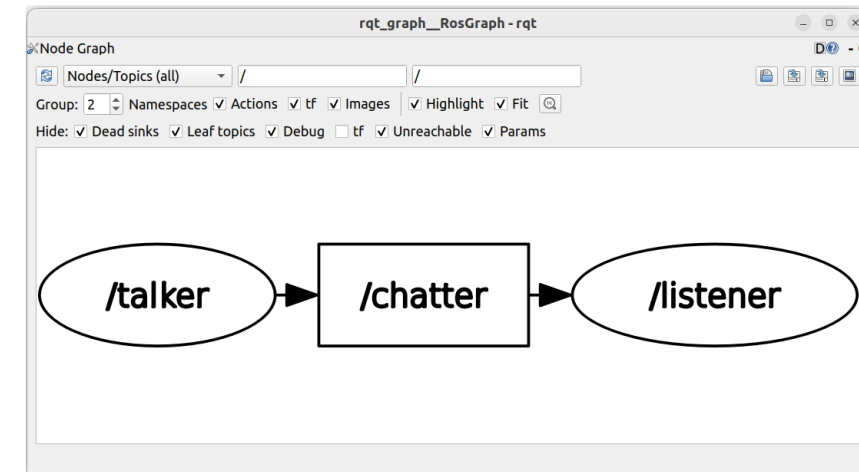
- In Terminal 2, input the following:

```
ros2 run demo_nodes_cpp listener
```

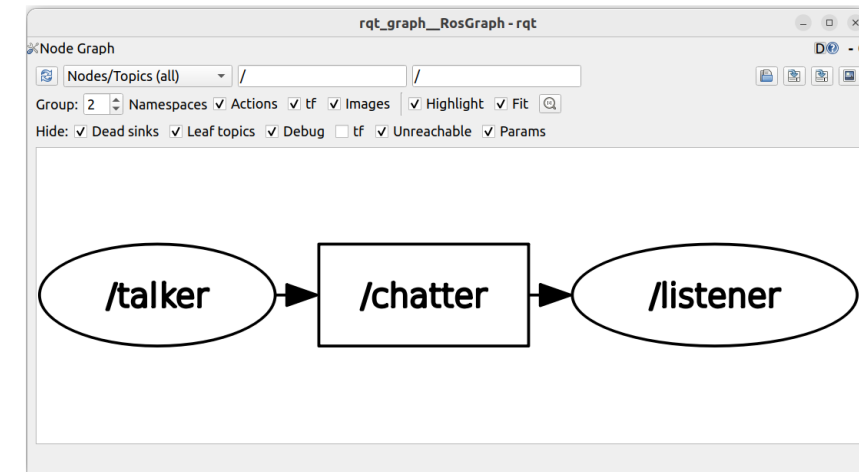
- In Terminal 3, input the following:

```
rqt_graph
```

- In the middle of the graph, we see a /chatter box. This box represents a **ROS 2 topic**.
- What we can also see is that the talker node is sending something to the /chatter topic, which will then be received by the listener node.
- We say that the talker is a publisher, and the listener is a subscriber.



- An important detail is that the talker is not actually sending data directly to the listener.
- The talker is publishing on the /chatter topic, and the listener is subscribing to the /chatter topic.
- Because of this, the data flows from the talker to the listener.



- From `rqt_graph`, we can already see that one node can send data to another node through a topic.
- An important point is that the topic is defined by a name. Both publishers and subscribers use the same name to make communication successful.
- There is more to it than just a name. Let's come back to the terminal and use the `ros2` command-line tool to discover more information.

- You previously used `ros2 run` to start a node. We also have `ros2 topic` to interact with topics.
- You can get more help with all available topic commands with **`ros2 topic -h`**.
- The `ros2 topic list` command will list all available topics, which means all topic communications between running nodes.

- In Terminal 3 (if you stopped `rqt_graph`), or in Terminal 4, run the following:
`ros2 topic list`
`/chatter`
`/parameter_events`
`/rosout`
- For any node we create, we will always see `/rosout` and `/parameter_events`. Those are not important for now and we can just ignore them.
- What's important is the `/chatter` topic. We already know it's used between the talker and listener node, but now the question is this: What kind of data is being sent?

- To get this information, we can use `ros2 topic info <topic_name>`:

`ros2 topic info /chatter`

Type: `std_msgs/msg/String`

Publisher count: 1

Subscription count: 1

- Here, we see how many nodes are publishing and subscribing to this topic. We have one publisher (talker node) and one subscriber (listener node).
- We can also see what kind of message is being sent: `std_msgs/msg/String`. In ROS 2, this message is called an **interface**.

- To see what's inside an interface, run `ros2 interface show <interface_name>`:

```
ros2 interface show std_msgs/msg/String
```

```
# Some comments
```

```
string data
```

- There can be a bunch of comments (starting with #) that you can ignore.
- The important thing is this: string data. This tells us what's being sent on that topic. Here, it's a string (chain of characters) with the name **data**.

- So, when the talker node wants to send a message to the /chatter topic, it needs to send a data field of type string.
- The listener, to get that information, will need to subscribe to /chatter, and expect to receive the same data type.
- That is how a topic is defined: a name and an interface (data type). Both publishers and subscribers should use the same name and interface to communicate.

- This makes sense: as an analogy, imagine you and I are talking through an online chat.
- If we are not in the same chat room (same topic name), we won't be able to find each other.
- Also, if I'm talking to you in a language you don't speak, this would not make sense to you.
- To communicate, we both need to agree on what language we use (same interface).

- Run the second example we did with the 2D turtlesim robot (two nodes).
- What I want you to do right now is to find the topic name and interface used by the turtle_teleop node to send a velocity command to the turtlesim node.
- Use the previous commands from this lecture to try to get that information.

- Start the two nodes and rqt_graph.
- In Terminal 1, input the following:
ros2 run turtlesim turtlesim_node
- In Terminal 2, input the following:
ros2 run turtlesim turtle_teleop_key
- In Terminal 3, input the following:
rqt_graph

- There are some more things on this screen, but we just need one piece of information.
- As you can see, there is a `/turtle1/cmd_vel` box, here representing a topic. The `teleop_turtle` node is a publisher, and the `turtlesim` node is a subscriber to that topic.
- This is quite logical: the `teleop_turtle` node will read the keys that you press, and then publish on the topic.
- On its end, the `turtlesim` node will subscribe to that topic to get the latest velocity command for the robot.

- We can get roughly the same information from the terminal:

ros2 topic list

/parameter_events

/rosout

/turtle1/cmd_vel

/turtle1/color_sensor

/turtle1/pose

- From the list of all running topics, we can spot the **/turtle1/cmd_vel** topic.

- Now, to retrieve the interface (data type) for that topic, run the following command:

ros2 topic info /turtle1/cmd_vel

Type: geometry_msgs/msg/Twist

Publisher count: 1

Subscription count: 1

- For the details of what's inside the interface, run the following command:

```
ros2 interface show geometry_msgs/msg/Twist
```

```
# This expresses velocity in free space broken into its linear and angular parts.
```

```
Vector3 linear
```

```
float64 x
```

```
float64 y
```

```
float64 z
```

```
Vector3 angular
```

```
float64 x
```

```
float64 y
```

```
float64 z
```

- This interface is a bit more complex than the one we had before. There's no need to make sense of all of that for now, as we will dive into interfaces later.
- The goal here is just to find a topic name and interface.
- From this information, let's say we want to make the robot move forward. We can guess that we would need to set a value for the x field, inside the linear field (as x is pointing forward in ROS).

- Nodes
- Topics
- **Services**
- Actions
- Parameters
- Launch Files

- Topics are very useful to send a stream of data/commands from one node to another node.
- However, this is not the only way to communicate. You can also find client/server communications in ROS 2. In this case, **services** will be used.
- As we did for topics, we will run two nodes communicating with each other, this time with services, and we will try to analyze, using the ROS 2 tools, what's happening and how the communication is working.

- Stop all running nodes.
- This time, we will start another node from `demo_nodes_cpp`, which contains a simple service server to add two integer numbers.
- We will also start a client node, which will send a request to the server node.

- In Terminal 1, input the following:

```
ros2 run demo_nodes_cpp add_two_ints_server
```

- In Terminal 2, input the following:

```
ros2 run demo_nodes_cpp add_two_ints_client
```

- As soon as you run the client node, you can see this log in Terminal 1 (server):

```
[INFO] [1710301834.789596504] [add_two_ints_server]: Incoming request  
a: 2 b: 3
```

- You can also see this log in Terminal 2 (client):

```
[INFO] [1710301834.790073100] [add_two_ints_client]: Result of add_  
two_ints: 5
```

- From what we observe here, it seems that the server node is hanging and waiting.
- The client node will send a request to the server, with two integer numbers, in this example: 2 and 3.
- The server node receives the request, adds the number, and returns the result: 5.

- Then, the client gets the response and prints the result.
- This is basically how a service works in ROS 2. You run one node that contains a server, then any other node (client) can send a request to that server.
- The server processes the request and returns a response to the client node.

- As for topics, services are defined by two things: a name, and an interface (data type).
- The only difference is that the interface will contain two parts: **a request and a response**.
- Unfortunately, `rqt_graph` does not support service introspection—although there are some plans to implement this in future ROS 2 distributions.

- To find the name of the service, we can use the `ros2` command-line tool again, this time with the `service` command, followed by `list`.
- As you can see, if you understand the way to list all topics, then it's exactly the same for services.
- At this point, you still have the service node running on Terminal 1, and nothing running on Terminal 2 (as the client stopped after receiving the response).

- In Terminal 2 or 3, run the following:

ros2 service list

/add_two_ints

/add_two_ints_server/describe_parameters

/add_two_ints_server/get_parameter_types

/add_two_ints_server/get_parameters

/add_two_ints_server/list_parameters

/add_two_ints_server/set_parameters

/add_two_ints_server/set_parameters_atomically

- That's a lot of services. Most of them can be discarded. For each node, you automatically get six additional services, all of them containing the name parameter.
- If we ignore them, we can see the `/add_two_ints` service, which is the service server running on the `add_two_ints_server` node.
- Great, we found the name. Now, to get the data type, we can use `ros2 service type <service_name>`, and then `ros2 interface show <interface_name>`:

ros2 service type /add_two_ints

example_interfaces/srv/AddTwoInts

ros2 interface show example_interfaces/srv/AddTwoInts

int64 a

int64 b

int64 sum

- We can see that the interface contains a line with **three dashes (---)**. This is the separation between the request and the response.
- With this, we know that to send a request to the server (as a client), you need to send one integer number named *a*, and another integer number named *b*.
- Then, we will receive a response containing one integer number named *sum*.

- Instead of running the `add_two_ints_client` node, we can also send a request directly from the terminal.
- This is important because it's a very useful way to test a service without requiring an existing client node.
- The syntax is `ros2 service call <service_name> <interface_name> "<request_in_json>"`.
- As we can see, we need to provide both the service name and interface.

- Here is an example of how to do that (make sure the server node is still running):

```
ros2 service call /add_two_ints example_interfaces/srv/AddTwoInts  
"{a: 4, b: 7}"
```

waiting for service to become available...

requester: making request: example_interfaces.srv.AddTwoInts_

Request(a=4, b=7)

response:

example_interfaces.srv.AddTwoInts_Response(sum=11)

- With this command, we send a request with 4 and 7.

- The server node will print those logs:

```
[INFO] [1710302858.634838573]  
[add_two_ints_server]: Incoming request  
a: 4 b: 7
```

- In the end, on the client side, we get the response that contains sum=11.

- Start the turtlesim node, list the existing services, and find how to spawn a new turtle robot in the 2D screen, using the terminal.
- Once again, we recommend you take a bit of time to try to do this on your own.
- Feel free to review all the previous commands from this lecture.

- Stop all running nodes.
- In Terminal 1, input the following:
ros2 run turtlesim turtlesim_node
- In Terminal 2, input the following:
ros2 service list
`/clear`
`/kill`
`/reset`
`/spawn`
`/turtle1/set_pen`
`/turtle1/teleport_absolute`
`/turtle1/teleport_relative`

- We already have the name; now, let's find the interface (request, response):

ros2 service type /spawn

turtlesim/srv/Spawn

ros2 interface show turtlesim/srv/Spawn

float32 x

float32 y

float32 theta

string name # Optional. A unique name will be created and returned if this is empty

string name

- Now, we have all the information we need.
- To send a request to the server, we have to use the /spawn service and the turtlesim/srv/Spawn interface.
- We can send a request that contains (x, y, theta) coordinates, plus an optional name.
- Actually, note that all fields in the request are optional. If you don't provide a value for a field, the default will be 0 for numbers, and "" for strings.

- Let's now send our request from the terminal:

```
ros2 service call /spawn turtlesim/srv/Spawn "{x: 3.0, y: 4.0}"
```

waiting for service to become available...

```
requester: making request: turtlesim.srv.Spawn_Request(x=3.0, y=4.0,  
theta=0.0, name="")
```

response:

```
turtlesim.srv.Spawn_Response(name='turtle2')
```

- If you look at the 2D window, you will see a new turtle. This turtle has been spawned at the (x, y, theta) coordinates provided in the request.



- Nodes
- Topics
- Services
- **Actions**
- Parameters
- Launch Files

- A ROS 2 action is basically the same thing as a service (client/server communication), but **designed for longer tasks**, and when you might want to **also get some feedback** during the execution, be able to cancel the execution, and so on.
- For example, making the robot move is not something that happens instantly. It could take a fraction of a second, but sometimes a task could take a few seconds/minutes or more.
- ROS 2 services have been designed for quick execution, for example: a computation, or an immediate action, such as spawning a turtle on a screen.

- Actions are used whenever a client/server communication might take more time and we want more control over it.

- Stop all running nodes, and start the turtlesim node again in Terminal 1:

```
ros2 run turtlesim turtlesim_node
```

- List all existing actions in Terminal 2:

```
ros2 action list
```

```
/turtle1/rotate_absolute
```

- From what we observe, it seems that the turtlesim node contains one action server.
- There is no existing client node for this action, so we will try to interact with it from the terminal. Of course, we will need two things: **the name and the interface.**

- As for topics and services, an action will be defined by a name and an interface. This time, the interface contains three parts: **goal, result, and feedback**.
- The goal and result are similar to the request and response for a service. The feedback is additional data that can be sent by the server to give some feedback during the goal execution.
- To get the action interface, you can run the `ros2 action info <action_name> -t` command. **Don't forget to add -t** (for type), otherwise, you'll see some details, but no interface:

```
ros2 action info /turtle1/rotate_absolute -t
```

```
Action: /turtle1/rotate_absolute
```

```
Action clients: 0
```

```
Action servers: 1
```

```
/turtlesim [turtlesim/action/RotateAbsolute]
```

- We can see that the action is running within one server (the turtlesim node), and we also found the interface: turtlesim/action/RotateAbsolute.

- Let's see what's inside this interface:

```
ros2 interface show turtlesim/action/RotateAbsolute
```

```
# The desired heading in radians
```

```
float32 theta
```

```
---
```

```
# The angular displacement in radians to the starting position
```

```
float32 delta
```

```
---
```

```
# The remaining rotation in radians
```

```
float32 remaining
```

- You can see two separations with three dashes (---).
The first part is the goal, the second part is the result, and the third part is the feedback.
- This action is quite simple; we only have one float number for each part of the interface.
- As a client, we send the desired angle for rotation. The server node will receive the goal and process it while optionally sending some feedback. When the goal is finished, the server will send the result to the client.

- As an action client, we are firstly interested in the goal part of the interface.
- Here, we need to send a float number, which corresponds to the angle (in radians) we want to rotate the turtle to.
- The syntax to send a goal from the terminal is `ros2 action send_goal <action_name> <action_interface> "<goal_in_json>"`. Once again, we need to provide both the name and interface.

- Make sure the turtlesim node is alive, then send a goal from Terminal 2:
**ros2 action send_goal /turtle1/rotate_absolute turtlesim/action/
RotateAbsolute "{theta: 1.0}"**

Waiting for an action server to become available...

Sending goal:

theta: 1.0

Goal accepted with ID: 3ba92096282a4053b552a161292afc8e

Result:

delta: -0.9919999837875366

Goal finished with status: SUCCEEDED

- After you run the command, you should see the turtle robot rotate on the 2D window.
- Once the desired angle is reached, the action will finish, and you will receive the result

- Nodes
- Topics
- Services
- Actions
- **Parameters**
- Launch Files

- We are now going to come back to the node itself and talk about another important ROS 2 concept: **parameters**.
- This time, it's not about communication, but about how to give different settings to a node when you start it.

- Stop all running nodes, and start the turtlesim node in Terminal 1:

ros2 run turtlesim turtlesim_node

- Then, to list all parameters, it's quite easy, and you can probably guess the command.
- If we have ros2 topic list for topics, ros2 service list for services, and ros2 action list for actions, then, for parameters, we have ros2 param list.
- The only particularity is that we use the word param instead of parameter.

- Run this command in Terminal 2:

ros2 param list

/turtlesim:

background_b

background_g

background_r

holonomic

qos_overrides./parameter_events.publisher.depth

qos_overrides./parameter_events.publisher.durability

qos_overrides./parameter_events.publisher.history

qos_overrides./parameter_events.publisher.reliability

start_type_description_service

use_sim_time

- We first see the turtlesim node, then a list of names under this node, with an indentation.
- Those names are the parameters, and they belong to the node. That's the first thing about parameters in ROS 2: they exist within a node. If you stop this turtlesim node, then the parameters would also be destroyed.
- There are a bunch of parameters you can ignore: `use_sim_time`, `start_type_description_service`, and all the parameters containing `qos_overrides`. Those will be present for any node you start.

- If we get rid of them, we are left with a few parameters, including `background_b`, `background_g`, `background_r`.
- From this observation, it seems that we would be able to change the background color of the 2D window when we start the `turtlesim` node.
- Now, what's inside those parameters? What kind of value? Is it a round number, a float, or a string?

- Let's find out, with `ros2 param get <node_name> <param_name>`.

- In Terminal 2, run the following commands:

```
ros2 param get /turtlesim background_b
```

Integer value is: 255

```
ros2 param get /turtlesim background_g
```

Integer value is: 86

```
ros2 param get /turtlesim background_r
```

Integer value is: 69

- From this, we can guess that the red, green, blue (RGB) value for the background is (69, 86, 255). It also seems that the parameter value is a round number from 0 to 255.
- Now that we have found the name of each parameter, and what kind of value we should use, let's modify the value ourselves when we start the node.
- For this, we will need to restart the node, using the same syntax as before: `ros2 run <package_name> <executable_name>`. We will then add `--ros-args` (only once), and `-p <param_name>:=value` for each parameter we want to modify.

- Stop the turtlesim node on Terminal 1, and start it again, with a different value for some of the parameters:

```
ros2 run turtlesim turtlesim_node --ros-args -p  
background_b:=0 -p background_r:=0
```

- Here, we decided that both the blue and red colors would be 0. We don't specify any value for background_g, which means that the default value will be used (as seen previously: 86).
- After you run this command, you should see the 2D screen appear, but this time, the background is dark green.

- Nodes
- Topics
- Services
- Actions
- Parameters
- **Launch Files**

- A launch file will allow you to **start several nodes and parameters from just one file**, which means that you can start your entire application with just one command line.
- To start a single node in the terminal, you have seen the `ros2 run` command. For launch files, we will use `ros2 launch <package_name> <launch_file>`.
- Stop all running nodes, and let's start the `talker_listener` launch file from the `demo_nodes_cpp` package.

- In Terminal 1, run the following command:

```
ros2 launch demo_nodes_cpp talker_listener_launch.py
```

```
[INFO] [launch]: All log files can be found below /home/ed/.ros/  
log/2024-03-14-16-09-27-384050-ed-vm-2867
```

```
[INFO] [launch]: Default logging verbosity is set to INFO
```

```
[INFO] [talker-1]: process started with pid [2868]
```

```
[INFO] [listener-2]: process started with pid [2871]
```

```
[talker-1] [INFO] [1710403768.481156318] [talker]: Publishing: 'Hello  
World: 1'
```

```
[listener-2] [INFO] [1710403768.482142732] [listener]: I heard: [Hello  
World: 1]
```

- As we can see, it seems that both the talker and listener nodes have been started.
- We can easily verify that in Terminal 2:
ros2 node list
/listener
/talker
- With rqt_graph, you could also check that the nodes communicate with each other.

- We have proof of that with the logs: on the same screen, we get both logs from the talker and listener nodes, and it seems that the listener node is receiving messages (using the /chatter topic as we saw previously).
- In the end, it's the same thing as if we had started both nodes on two terminals. The launch file will simply start the two nodes in one terminal.
- If we read the logs more carefully, we can see that each node will be started in a different process. To stop the launch file, press Ctrl + C. This will stop all processes (nodes), and your application will end.

- Let's now try another launch file from the turtlesim package. Stop the launch file in Terminal 1, and start the multisim launch file from the turtlesim package:

```
ros2 launch turtlesim multisim.launch.py
```

```
[INFO] [launch]: All log files can be found below /home/fahad/  
.ros/log/2025-04-02-16-14-41-043618-fahad-VirtualBox-2962
```

```
[INFO] [launch]: Default logging verbosity is set to INFO
```

```
[INFO] [turtlesim_node-1]: process started with pid [2963]
```

```
[INFO] [turtlesim_node-2]: process started with pid [2965]
```

- With this, you will see not one, but two 2D windows, containing each a turtle robot.
- As we can see from the logs, we are starting two turtlesim nodes (two identical nodes with a different name each).
- We can also check that from the terminal:

ros2 node list

/turtlesim1/turtlesim

/turtlesim2/turtlesim

Thank you for your attention.

Any Questions?

